```
using System;
namespace CityBreaks.Services
{
    public class LifetimeDemoService
    {
        public LifetimeDemoService()
        {
            Value = Guid.NewGuid();
        }
        public Guid Value { get; }
    }
}
```

The public `Value` property is set whenever the class constructor is called, which is when the container instantiates the service. You will register this service as a singleton, which should ensure it is only ever instantiated once during the life of the application:

```
builder.Services.AddRazorPages();
builder.Services.AddTransient<ICityService, SimpleCityService>();
builder.Services.AddSingleton<LifetimeDemoService>();
```

The service is registered with a version of the `AddSingleton` method that takes a single generic parameter representing the implementation. You don't have an abstraction for this example. It's not needed because this is a simple demonstration, and an abstraction will provide an unnecessary distraction from the main point of the examples that follow. Create a new Razor page called LifetimeDemo.cshtml in the Pages folder with the following code in the `PageModel` class file.

Listing 7.7 The `LifetimeDemoModel` code for demonstrating how service lifetimes work

```
using CityBreaks.Services;
using Microsoft.AspNetCore.Mvc.RazorPages;

namespace CityBreaks.Pages
{
    public class LifetimeDemoModel : PageModel
    {
```